

Henrique de Paula Balbino
Pedro Lucas Oliveira Comper
Luan Araújo de Oliveira

Projeto do Bimestre: Introdução aos Sistemas Operacionais e ao Linux

Vila Velha – ES
2026

Henrique de Paula Balbino
Pedro Lucas Oliveira Comper
Luan Araújo de Oliveira

Projeto do Bimestre: Introdução aos Sistemas Operacionais e ao Linux

Projeto do bimestre — Destinação Linux.

Universidade Vila Velha
Campus de Boa Vista
Ciência da Computação
Disciplina: Sistemas Operacionais
Professor: Jean-Rémi Bourguet
Período: 2026/2

Vila Velha – ES
2026

Resumo

Este trabalho introduz conceitos fundamentais de sistemas operacionais a partir das relações históricas e técnicas entre UNIX, GNU e Linux. Também discute sistemas macOS e Windows, distribuições, interfaces de linha de comando e gerenciamento de pacotes. A parte final organiza as atividades requeridas do livro PEIXOTO e BONFANTE (2013); como as páginas correspondentes não foram disponibilizadas, ela preserva a estrutura de resolução e registra precisamente as informações ainda necessárias. **Palavras-chave:**

sistemas operacionais; GNU/Linux; shell; terminal.

Sumário

1	INTRODUÇÃO	4
2	UNIX	5
3	GNU	6
4	LINUX	7
5	MACOS E WINDOWS	8
6	DISTRIBUIÇÕES LINUX	9
7	TERMINAIS E SHELLS	10
8	GERENCIAMENTO DE PACOTES	11
9	COMANDOS	12
10	ORGANIZAÇÃO DO LINUX	13
10.1	Exercício de fixação 1 (página 40)	13
10.2	Atividade 3.2 (página 61)	14
10.3	Atividade 3.3 (página 61)	14
10.4	Atividade 3.4 (página 61)	15
10.5	Atividade 3.5 (página 61)	16
11	DESVENDANDO O LINUX	18
11.1	Exercício de fixação 1 (página 68)	18
11.2	Atividade 4.1 (página 77)	19
11.3	Atividade 4.2 (página 77)	19
11.4	Atividade 4.3 (página 77)	20
11.5	Atividade 4.4 (página 77)	20
12	CONCLUSÃO	24
	REFERÊNCIAS	25

1 Introdução

Um sistema operacional media o uso de processador, memória, armazenamento e dispositivos por programas concorrentes. Mais que uma interface gráfica, ele define abstrações—processos, arquivos, usuários e permissões—que permitem compartilhar recursos com isolamento. Este texto progride da linhagem UNIX até práticas básicas de uso do Linux; a Seção 9 reúne exemplos práticos.

2 UNIX

UNIX surgiu nos Bell Labs no fim da década de 1960, no contexto posterior ao projeto Multics, e consolidou ideias decisivas: processos pequenos combináveis, sistema de arquivos hierárquico e interface textual. A reescrita predominante em C favoreceu a portabilidade entre arquiteturas. Sua influência permanece nas interfaces POSIX e na organização de sistemas contemporâneos; Linux busca compatibilidade com essas convenções (Linux Kernel Organization, 2024).

Sua filosofia costuma ser sintetizada em programas que executam bem uma tarefa e que podem ser compostos. Nessa composição, a saída textual de um programa pode se tornar a entrada de outro por meio de um *pipe*. Essa abordagem reduz acoplamento e torna tarefas complexas reproduzíveis a partir de ferramentas simples. Em termos de arquitetura, programas no espaço do usuário solicitam serviços ao kernel, que administra recursos e dispositivos; o shell interpreta os comandos do usuário e inicia esses programas.

3 GNU

O Projeto GNU foi anunciado em 1983 com a meta de construir um sistema operacional livre compatível com UNIX (Free Software Foundation, 2026). “Livre” refere-se às liberdades de usar, estudar, modificar e redistribuir o software, e não necessariamente ao preço. Compilador, bibliotecas, shell e ferramentas GNU compõem grande parte do ambiente de usuário normalmente chamado de GNU/Linux.

Essas liberdades abrangem executar o programa para qualquer finalidade; estudar seu funcionamento e adaptá-lo; redistribuir cópias; e distribuir versões aperfeiçoadas. O acesso ao código-fonte é condição prática para estudar e modificar, mas a ênfase do software livre é a liberdade do usuário. Entre componentes relevantes do ecossistema estão GCC, Emacs, GDB e Coreutils. Assim, GNU não é apenas uma licença: é também um conjunto de ferramentas que viabiliza o uso cotidiano de sistemas livres.

4 Linux

Linux é, estritamente, o núcleo: gerencia escalonamento, memória, dispositivos e chamadas de sistema. Uma instalação utilizável reúne esse núcleo com programas de espaço do usuário. O próprio kernel.org distingue o kernel de uma distribuição completa (Linux Kernel Organization, 2024). A Figura 1 relaciona marcos dessa composição.

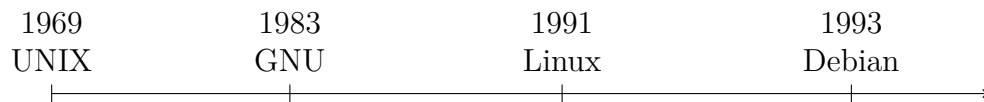


Figura 1 – Linha do tempo resumida da linhagem UNIX, GNU e Linux.

Definição 4.1 (Kernel). *O kernel é a parte privilegiada do sistema operacional que arbitra o acesso ao hardware e oferece serviços aos programas por meio de chamadas de sistema.*

Em particular, a execução de um processo alterna entre código em modo usuário e serviços requisitados ao kernel, conforme a Definição 4.1.

O kernel Linux foi iniciado por Linus Torvalds em 1991 e se beneficiou de uma comunidade distribuída de desenvolvimento. A combinação do kernel com ferramentas GNU tornou possível um ambiente operacional completo; por isso, a denominação GNU/Linux ressalta a participação dessas ferramentas, enquanto “Linux” é frequentemente usado de forma abreviada para todo o sistema. Essa distinção evita confundir núcleo, sistema operacional e distribuição.

5 macOS e Windows

macOS é um sistema proprietário da Apple, cuja base Darwin integra tecnologias BSD e Mach (Apple, 2013). Windows é a família de sistemas da Microsoft organizada em componentes de modo usuário e modo kernel (Microsoft, 2026). Não se deve confundir interface gráfica com arquitetura: ambos oferecem GUIs, mas também expõem mecanismos de processos, memória e E/S. Uma diferença relevante para o estudo é que macOS deriva de uma base UNIX, enquanto o Windows usa a arquitetura Windows NT e pode oferecer compatibilidade Linux por subsistemas como WSL.

Em macOS, o ambiente Darwin disponibiliza componentes e convenções UNIX, incluindo ferramentas de linha de comando, embora a experiência usual do usuário seja gráfica. No Windows, programas de modo usuário dependem de serviços oferecidos pelo sistema, enquanto drivers e gerenciadores fundamentais operam em modo kernel. Em ambos os casos, a interface gráfica não elimina a necessidade de gerenciamento de processos, memória, arquivos e dispositivos.

6 Distribuições Linux

Uma distribuição integra kernel, instalador, repositórios, gerenciador de pacotes, configuração e aplicações. Portanto, não é sinônimo de Linux. A Tabela 1 compara famílias sem sugerir superioridade absoluta. Além do formato de pacote, as distribuições diferem

Tabela 1 – Exemplos de famílias de distribuições.

Família	Gerenciador usual	Característica geral
Debian	APT	estabilidade e amplo repositório
Fedora/RHEL	DNF	ecossistema RPM
Arch	pacman	atualização contínua e configuração explícita

pela política de atualização, pelo conjunto de programas padrão, pelo suporte comunitário ou empresarial e pelas decisões de configuração. Debian privilegia estabilidade e ampla disponibilidade de pacotes; Fedora costuma apresentar tecnologias recentes; Arch privilegia instalação enxuta e atualizações contínuas. A escolha adequada depende do objetivo, como desktop, servidor, ensino, desenvolvimento ou administração de segurança.

7 Terminais e shells

O terminal é o dispositivo ou emulador que apresenta entrada e saída textual; o shell é o interpretador que lê comandos e inicia programas. Bash, Zsh e Fish são shells. Essa separação explica por que trocar o emulador de terminal não altera necessariamente os comandos disponíveis. O shell ainda viabiliza composição com redirecionamentos e pipes, cujo valor está em ligar a saída de um programa à entrada de outro.¹

Também é útil distinguir console, terminal e pseudo-terminal. Um console é uma interface diretamente associada ao sistema; um emulador gráfico cria um pseudo-terminal para receber entrada e apresentar saída. O comando `tty` informa o dispositivo associado à sessão atual. Já o *prompt* sinaliza que o shell está pronto para receber um comando e é justamente o elemento que deve identificar um integrante nas capturas solicitadas pelo professor.

O Bourne shell (`sh`) estabeleceu uma sintaxe influente em UNIX. O Bash, desenvolvido no âmbito do GNU, é largamente empregado em distribuições Linux; Zsh e Fish oferecem alternativas com recursos próprios. Embora os três possam iniciar comandos, scripts e configurações não são automaticamente portáveis entre eles.

¹ O pipe é representado por `|`; não cria um arquivo intermediário por si só.

8 Gerenciamento de pacotes

Gerenciadores resolvem dependências, verificam metadados e instalam versões rastreáveis por repositórios. O administrador deve atualizar índices e pacotes segundo a política da distribuição, evitando misturar repositórios incompatíveis. A quantidade de trabalho de atualização pode ser expressa, de forma abstrata, por

$$T_{atualizacao} = T_{download} + T_{verificacao} + T_{instalacao}. \quad (8.1)$$

A Equação 8.1 não é uma medição: apenas decompõe conceitualmente etapas do processo.

Em sistemas Debian, **dpkg** manipula diretamente pacotes **.deb**, enquanto APT opera em nível mais alto, consulta repositórios e resolve dependências. Nas famílias Red Hat e Fedora, RPM é o formato tradicional e DNF é uma ferramenta de alto nível. Há ainda formatos voltados a múltiplas distribuições, como Snap e Flatpak; eles facilitam a distribuição de aplicações, mas não substituem necessariamente o gerenciador nativo do sistema. A Tabela 1 mostra como esse componente se relaciona às famílias de distribuições.

9 Comandos

Comandos devem ser usados com atenção ao diretório atual, às permissões e aos argumentos. O Listing 9.1 ilustra consulta e manipulação não destrutiva de arquivos.

```
pwd # exibe o diretório atual  
ls -la # lista inclusive entradas ocultas  
mkdir -p dados/logs # cria a hierarquia solicitada  
find dados -type f # localiza arquivos regulares
```

Listing 9.1 – Exemplos básicos de navegação e inspeção.

- **pwd** reduz ambiguidades sobre o local em que comandos atuam;
- **ls** inspeciona entradas, mas não substitui critérios de busca;
- **find** seleciona arquivos por condições, como tipo ou nome.

Outros comandos recorrentes são **cp**, para copiar; **mv**, para mover ou renomear; **rm**, para remover; e **tar**, para empacotar arquivos. Caracteres-curinga também são importantes: ***** representa uma sequência de caracteres, **?** representa um único caractere e colchetes especificam alternativas. Como comandos de remoção podem afetar dados permanentemente, recomenda-se sempre confirmar o diretório atual com **pwd** e listar os alvos antes de executar uma remoção recursiva. O gráfico da Figura 2 contabiliza apenas os quatro comandos do Listing 9.1; é uma visualização didática, não dado empírico externo.

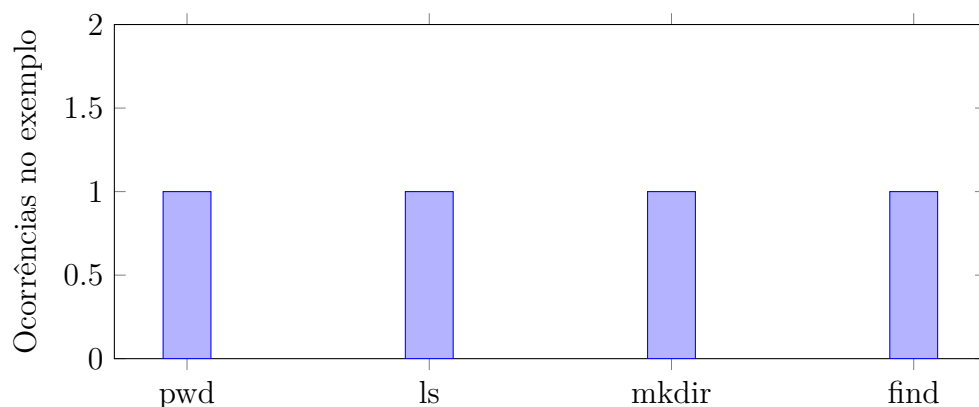


Figura 2 – Barplot das ocorrências de comandos no Listing 9.1.

10 Organização do Linux

Conforme o Capítulo 3 de PEIXOTO e BONFANTE (2013), a organização do Linux é hierárquica e tem a raiz / como ponto de partida. As respostas a seguir usam o diretório pessoal do integrante; substitua o prompt pelas informações reais do computador antes de produzir as capturas.

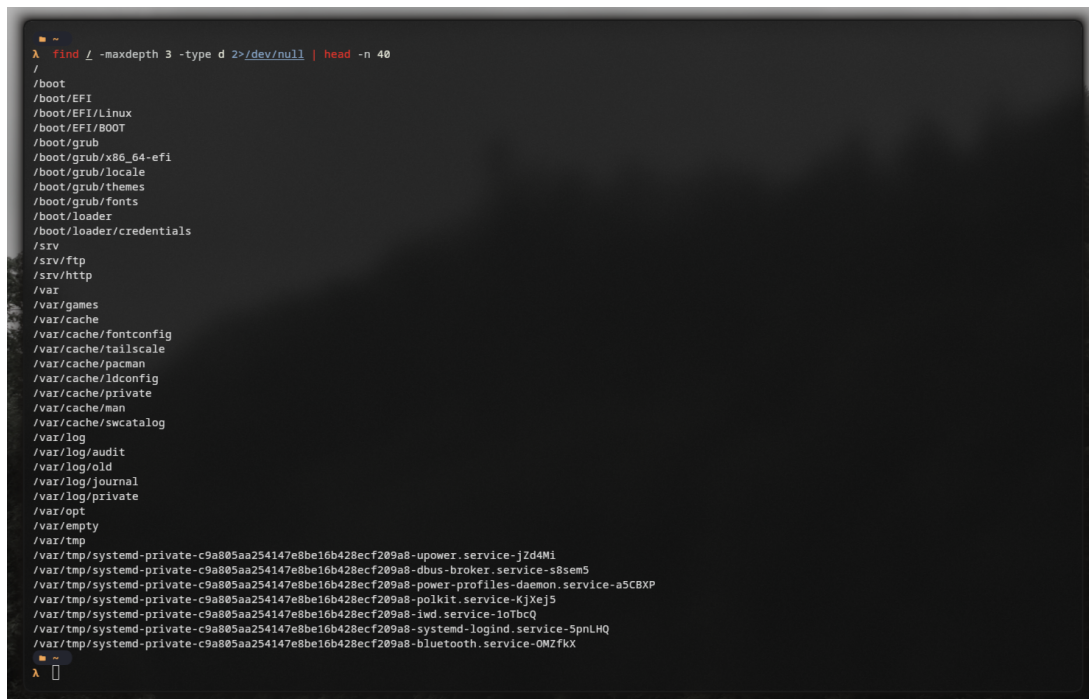
10.1 Exercício de fixação 1 (página 40)

Enunciado resumido. Desenhar a árvore do sistema de arquivos até o terceiro nível, considerando / como primeiro nível e mostrando no máximo três diretórios no terceiro nível.

Resolução. A estrutura inicia em /; no segundo nível, por exemplo, aparecem /bin, /etc e /home. Os diretórios do terceiro nível variam de acordo com a distribuição e devem ser anotados a partir do resultado efetivamente obtido. O comando do Listing 10.1 permite observá-los sem alterar o sistema.

```
find / -maxdepth 3 -type d 2>/dev/null | head -n 40
```

Listing 10.1 – Inspeção da hierarquia para o Exercício 3.1.



```

~
λ find / -maxdepth 3 -type d 2>/dev/null | head -n 40
/
/boot
/boot/EFI
/boot/EFI/Linux
/boot/EFI/BOOT
/boot/grub
/boot/grub/x86_64-efi
/boot/grub/locale
/boot/grub/themes
/boot/grub/fonts
/boot/loader
/boot/loader/credentials
/srv
/srv/ftp
/srv/http
/var
/var/games
/var/cache
/var/cache/fontconfig
/var/cache/tailscale
/var/cache/pacman
/var/cache/ldconfig
/var/cache/private
/var/cache/man
/var/cache/swcatalog
/var/log
/var/log/audit
/var/log/old
/var/log/journal
/var/log/private
/var/opt
/var/empty
/var/tmp
/var/tmp/systemd-private-c9a805aa254147e8be16b428ecf209a8-upower.service-jZd4M1
/var/tmp/systemd-private-c9a805aa254147e8be16b428ecf209a8-dbus-broker.service-s8sem5
/var/tmp/systemd-private-c9a805aa254147e8be16b428ecf209a8-power-profiles-daemon.service-a5CBXP
/var/tmp/systemd-private-c9a805aa254147e8be16b428ecf209a8-polkit.service-kjXej5
/var/tmp/systemd-private-c9a805aa254147e8be16b428ecf209a8-ivd.service-1oTbcQ
/var/tmp/systemd-private-c9a805aa254147e8be16b428ecf209a8-systemd-logind.service-5pnlHQ
/var/tmp/systemd-private-c9a805aa254147e8be16b428ecf209a8-bluetooth.service-QMzfkX
λ

```

Figura 3 – Inspeção da árvore de diretórios para o Exercício de fixação 1 do Capítulo 3.

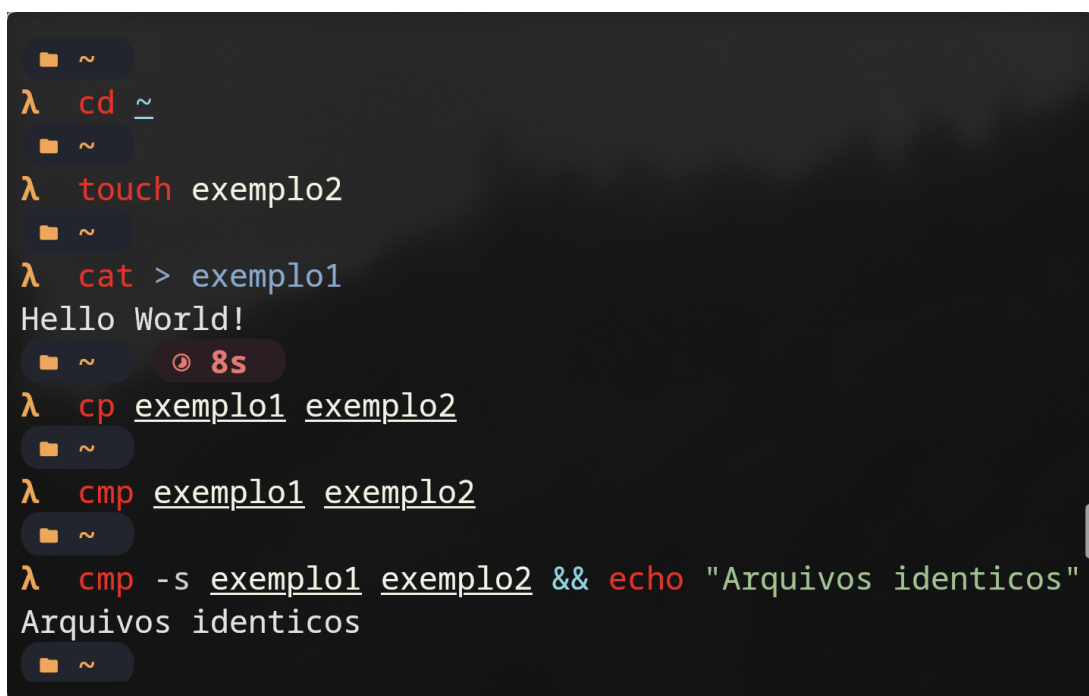
10.2 Atividade 3.2 (página 61)

Enunciado resumido. Criar `exemplo1` e `exemplo2` por comandos distintos, inserindo linhas pelo teclado em um deles, e tornar o arquivo vazio idêntico ao que possui conteúdo.

Resolução e raciocínio. `touch` cria o arquivo vazio. Já `cat >` lê as linhas digitadas até `Ctrl+D`; `cp` reproduz o conteúdo no segundo arquivo. A Figura 4 deve mostrar também as linhas digitadas e o resultado de `cmp`.

```
cd ~
touch exemplo2
cat > exemplo1
# digite algumas linhas e pressione Ctrl+D
cp exemplo1 exemplo2
cmp exemplo1 exemplo2
```

Listing 10.2 – Criação e comparação dos arquivos da Atividade 3.2.



```
λ cd ~
λ touch exemplo2
λ cat > exemplo1
Hello World!
λ cp exemplo1 exemplo2
λ cmp exemplo1 exemplo2
λ cmp -s exemplo1 exemplo2 && echo "Arquivos idênticos"
Arquivos idênticos
```

Figura 4 – Criação e cópia dos arquivos da Atividade 3.2.

10.3 Atividade 3.3 (página 61)

Enunciado resumido. Criar `/temp/exemplos` com um só comando e transferir os arquivos, mantendo uma cópia de `exemplo2` na origem.

Resolução e raciocínio. A opção `-p` cria os níveis ausentes. `mv` transfere `exemplo1`; `cp` coloca uma cópia de `exemplo2` no destino e preserva seu original. O resultado deve ser conferido na Figura 5.

```
mkdir -p ~/temp/exemplos
mv ~/exemplo1 ~/temp/exemplos/
cp ~/exemplo2 ~/temp/exemplos/
ls -l ~ ~/temp/exemplos
```

Listing 10.3 – Diretórios e cópia solicitados na Atividade 3.3.

```
λ mkdir -p ~/temp/exemplos
λ mv ~/exemplo1 ~/temp/exemplos/
λ cp ~/exemplo2 ~/temp/exemplos/
λ ls -l ~ ~/temp/exemplos

/home/henrique:
Permissions Size User Date Modified Name
drwxr-xr-x - henrique 16 Aug 13:02 ▀ _build
drwxr-xr-x - henrique 24 Sep 22:17 ▀ Desktop
drwxr-xr-x - henrique 9 Aug 18:10 ▀ Documents
drwxr-xr-x - henrique 24 Sep 22:17 ▀ Downloads
drwxr-xr-x - henrique 8 Aug 20:21 ▀ go
drwxr-xr-x - henrique 7 Sep 17:59 ▀ Microsoft
drwxr-xr-x - henrique 1 Aug 12:18 ▀ Music
drwxr-xr-x - henrique 5 Sep 15:25 ▀ Pictures
drwxr-xr-x - henrique 16 Aug 13:27 ▀ Postman
drwxr-xr-x - henrique 1 Aug 12:18 ▀ Projects
drwxr-xr-x - henrique 1 Aug 12:18 ▀ Public
drwxr-xr-x - henrique 16 Aug 12:04 ▀ revert
drwxr-xr-x - henrique 1 Aug 17:25 ▀ rofi
drwxr-xr-x - henrique 2 Aug 17:00 ▀ spicetify-themes
drwxr-xr-x - henrique 24 Sep 22:35 ▀ temp
drwxr-xr-x - henrique 1 Aug 12:18 ▀ Templates
drwxr-xr-x - henrique 29 Aug 11:06 ▀ Videos
drwxr-xr-x - henrique 1 Aug 10:59 ▀ yay
-rw-r--r-- 13 henrique 24 Sep 22:32 ▀ exemplo2
-rw-r--r-- 5.7M henrique 8 Aug 19:51 ▀ help
-rw-r--r-- 759 henrique 1 Aug 16:34 ▀ install.log

/home/henrique/temp/exemplos:
Permissions Size User Date Modified Name
-rw-r--r-- 13 henrique 24 Sep 22:32 ▀ exemplo1
-rw-r--r-- 13 henrique 24 Sep 22:35 ▀ exemplo2
```

Figura 5 – Criação de diretórios e transferência de arquivos da Atividade 3.3.

10.4 Atividade 3.4 (página 61)

Enunciado resumido. Empacotar e compactar os arquivos em `exemplos.tar.gz`, movê-lo para `exemplos2` e, nesse local, descompactá-lo e desempacotá-lo com um comando.

Resolução e raciocínio. Em `tar`, `-z` seleciona `gzip`, `-c` cria, `-v` exibe os nomes e `-f` define o arquivo. Na extração, `-x` substitui `-c`. A execução correspondente é a da Figura 6.

```
cd ~/temp/exemplos
tar -zcvf exemplos.tar.gz exemplo1 exemplo2
mkdir exemplos2
mv exemplos.tar.gz exemplos2/
cd exemplos2
tar -zxvf exemplos.tar.gz
```

Listing 10.4 – Empacotamento e extração da Atividade 3.4.

```
■ ~
λ cd ~/temp/exemplos
■ ~/temp/exemplos
λ tar -zcvf exemplos.tar.gz exemplo1 exemplo2
exemplo1
exemplo2
■ ~/temp/exemplos
λ mkdir exemplos2
■ ~/temp/exemplos
λ mv exemplos.tar.gz exemplos2/
■ ~/temp/exemplos
λ cd exemplos2
■ ../temp/exemplos/exemplos2
λ tar -zxvf exemplos.tar.gz
exemplo1
exemplo2
■ ../temp/exemplos/exemplos2
λ
```

Figura 6 – Empacotamento, compactação e extração da Atividade 3.4.

10.5 Atividade 3.5 (página 61)

Enunciado resumido. Após as atividades, retornar à situação original e remover os elementos criados.

Resolução e raciocínio. Antes de remover, `ls` confirma os alvos. `rm -r` remove a árvore criada; use-o somente com os caminhos explícitos do Listing 10.5. A Figura 7 deve mostrar a confirmação final.

```
ls -ld ~/temp ~/exemplo2
rm -r ~/temp
rm ~/exemplo2
ls -ld ~/temp ~/exemplo2
```

Listing 10.5 – Limpeza dos elementos criados na Atividade 3.5.


```
■ ../temp/exemplos/exemplos2
λ ls -ld ~/temp ~/exemplo2
Permissions Size User Date Modified Name
drwxr-xr-x - henrique 24 Sep 22:35 /home/henrique/temp
.rw-r--r-- 13 henrique 24 Sep 22:32 /home/henrique/exemplo2
■ ../temp/exemplos/exemplos2
λ rm -r ~/temp
■ ../temp/exemplos/exemplos2
λ rm ~/exemplo2
■ ../temp/exemplos/exemplos2
λ ls -ld ~/temp ~/exemplo2
"/home/henrique/temp": No such file or directory (os error 2)
"/home/henrique/exemplo2": No such file or directory (os error 2)
■ ../temp/exemplos/exemplos2
λ cd
■ ~
λ
```

Figura 7 – Remoção dos elementos criados na Atividade 3.5.

11 Desvendando o Linux

O Capítulo 4 apresenta filtros, redirecionamentos e canalizações (PEIXOTO; BONFANTE, 2013). Os comandos abaixo devem ser executados com os arquivos indicados pelo roteiro antes da captura.

11.1 Exercício de fixação 1 (página 68)

Enunciado resumido. Explicar a utilidade de `sort`, identificar a opção que ordena por coluna e pesquisar três opções.

Resolução. `sort` reordena linhas de entrada. A opção `-k` escolhe o campo (coluna); `-n` compara numericamente, `-r` inverte a ordem e `-k` seleciona a chave de ordenação. A página de manual na Figura 8 documenta essas opções.

```
man sort
# dentro do manual, pesquise por -k, -n e -r digitando /-k
```

Listing 11.1 – Consulta ao manual para o Exercício 4.1.



```
~  @ 1m22s
λ man sort | col -b | grep -E '^[:space:]*(-k|--key|-n|--numeric-sort|-r|--reverse)'
-n, --numeric-sort
-r, --reverse
-k, --key=KEYDEF
~
λ
```

Figura 8 – Consulta às opções do comando `sort` no Exercício de fixação 1 do Capítulo 4.

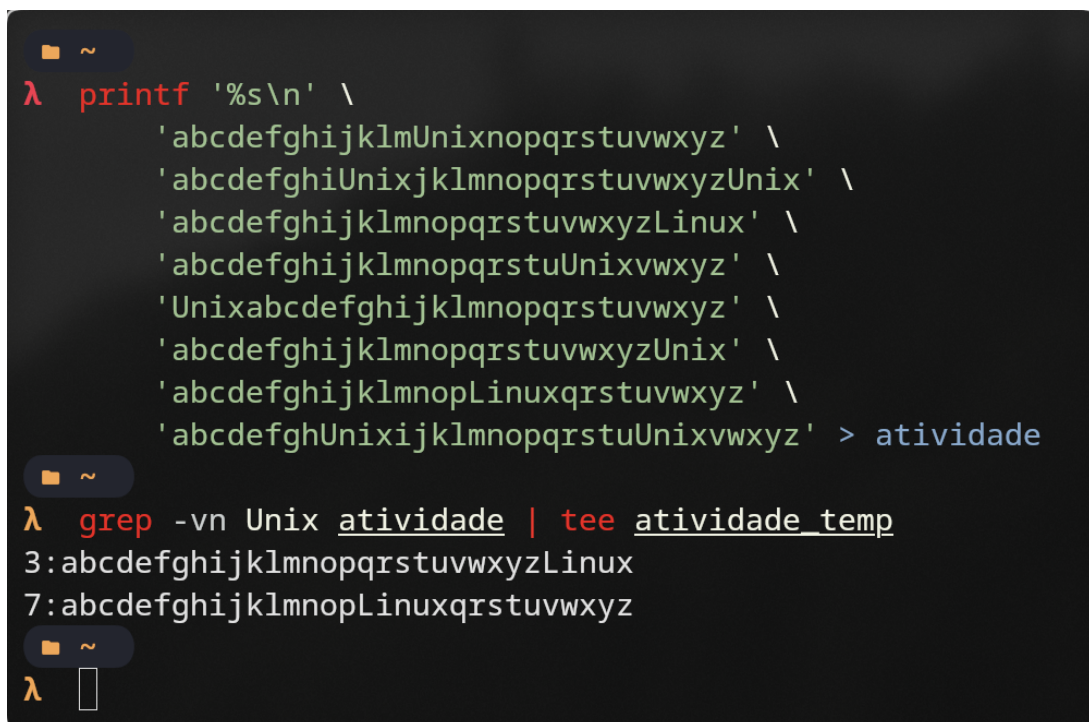
11.2 Atividade 4.1 (página 77)

Enunciado resumido. No arquivo `atividade` fornecido pelo roteiro, exibir com numeração as linhas sem a palavra `Unix` e salvar o resultado em `atividade_temp`.

Resolução e raciocínio. `grep -v` seleciona linhas que não contêm o padrão; `-n` acrescenta a numeração. `tee` mostra a saída e a grava simultaneamente. Devem aparecer as linhas 3 e 7, como evidenciado na Figura 9.

```
grep -vn Unix atividade | tee atividade_temp
```

Listing 11.2 – Pesquisa e gravação da Atividade 4.1.



```
λ printf '%s\n' \
    'abcdefghijklmUnixnopqrstuvwxyz' \
    'abcdefghiUnixjklmnopqrstuvwxyzUnix' \
    'abcdefghijklmnopqrstuvwxyzLinux' \
    'abcdefghijklmnopqrstuUnixvwxyz' \
    'Unixabcdefghijklmnopqrstuvwxyz' \
    'abcdefghijklmnopqrstuvwxyzUnix' \
    'abcdefghijklmnopLinuxqrstuvwxyz' \
    'abcdefghUnixijklmnopqrstuUnixvwxyz' > atividade
λ grep -vn Unix atividade | tee atividade_temp
3:abcdefghijklmnopqrstuvwxyzLinux
7:abcdefghijklmnopLinuxqrstuvwxyz
λ
```

Figura 9 – Linhas sem a palavra `Unix` na Atividade 4.1.

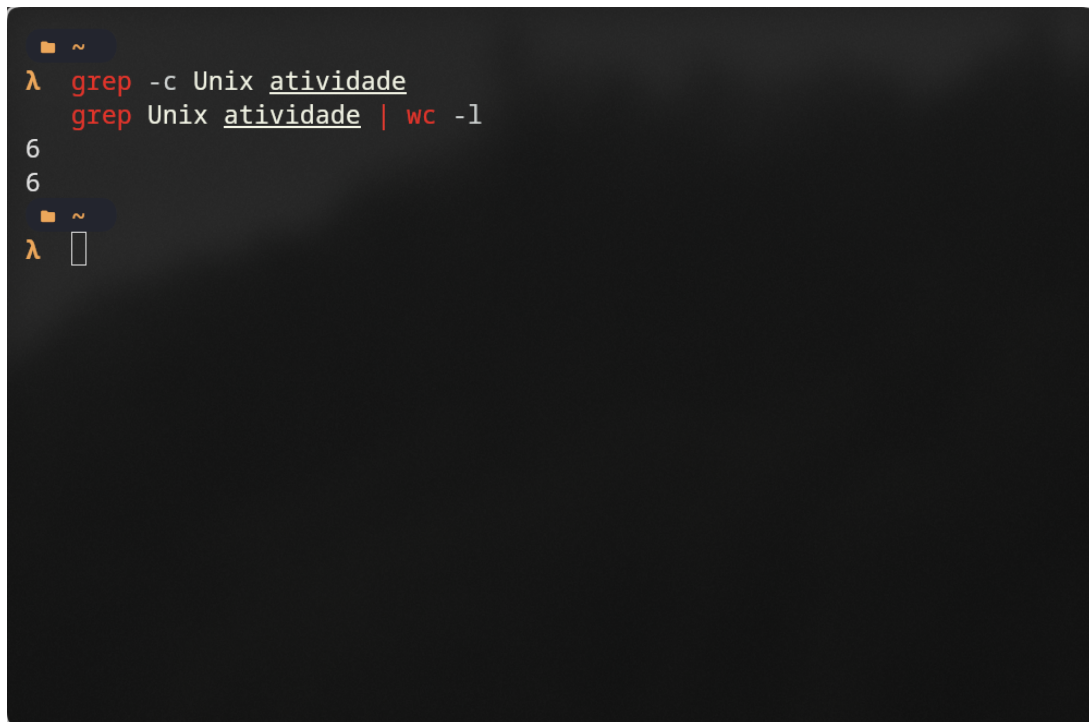
11.3 Atividade 4.2 (página 77)

Enunciado resumido. Mostrar, por mais de uma forma e sem listar as linhas, quantas linhas de `atividade` contêm ao menos uma ocorrência de `Unix`.

Resolução e raciocínio. `grep -c` conta diretamente as linhas correspondentes. A canalização alternativa seleciona as linhas e `wc -l` as conta. Ambas devem retornar 6, conforme a Figura 10.

```
grep -c Unix atividade
grep Unix atividade | wc -l
```

Listing 11.3 – Duas contagens para a Atividade 4.2.



```
~  
λ grep -c Unix atividade  
grep Unix atividade | wc -l  
6  
6  
~  
λ
```

Figura 10 – Contagem de linhas contendo Unix na Atividade 4.2.

11.4 Atividade 4.3 (página 77)

Enunciado resumido. Usar uma canalização para exibir apenas a terceira e a quarta linhas de `atividade`.

Resolução e raciocínio. `head -4` limita a entrada às quatro primeiras linhas, e `tail -2` retém as duas últimas desse conjunto. Assim, a Figura 11 mostra precisamente as linhas 3 e 4.

```
head -4 atividade | tail -2
```

Listing 11.4 – Canalização para a Atividade 4.3.

11.5 Atividade 4.4 (página 77)

Enunciado resumido. Reproduzir progressivamente, sobre o arquivo `clientes`, a ordenação numérica crescente solicitada no roteiro, usando canalização e redirecionamento, sem os comandos proibidos.

Resolução e raciocínio. O arquivo do roteiro está em ordem decrescente. Em cada etapa, a última linha é colocada depois do prefixo já organizado; o trecho intermediário é reconstruído por `tail` e `head`. Execute cada bloco, confira `clientes` com `less clientes` e somente então passe ao próximo. A solução do Listing 11.5 usa apenas `head`, `tail`, `mv`, pipes e redirecionamentos; não emprega os comandos vedados. A Figura 12 deve registrar as etapas iniciais, e a Figura 13, a ordenação concluída.



```
~  
λ head -4 atividade | tail -2  
abcdefghijklmnopqrstuvwxLinux  
abcdefghijklmnopqrstuUnixvwxyz  
~  
λ
```

Figura 11 – Terceira e quarta linhas do arquivo atividade na Atividade 4.3.

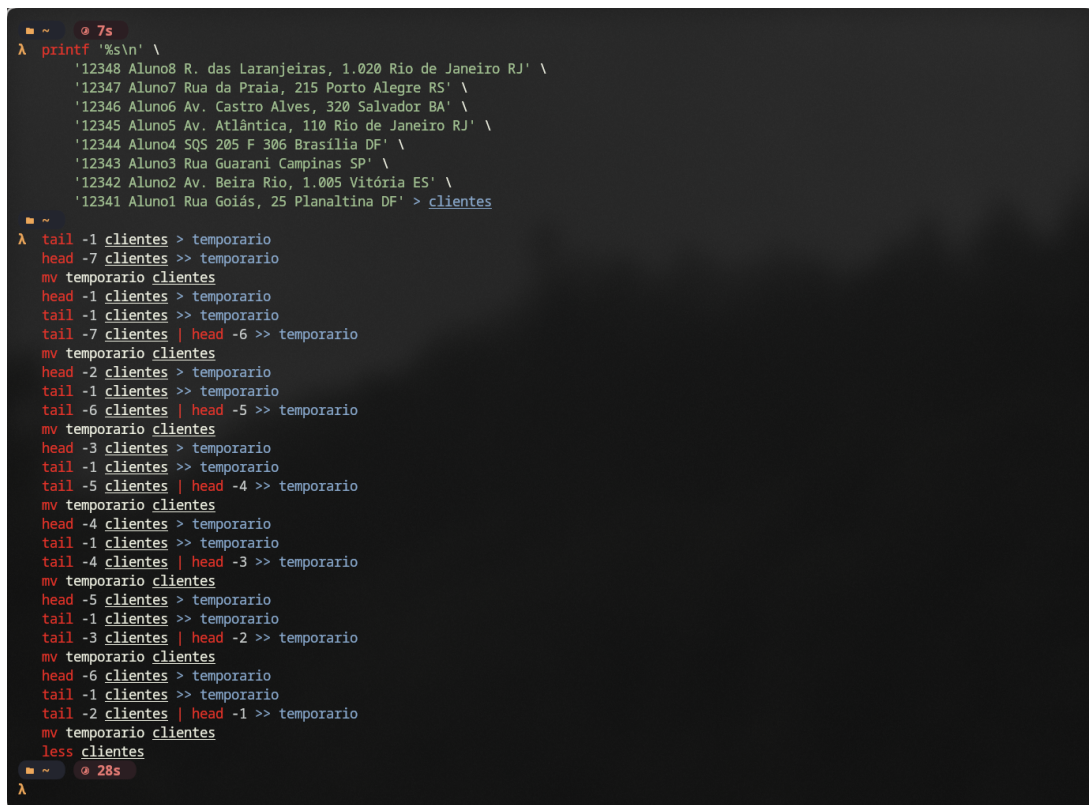
```
tail -1 clientes > temporario  
head -7 clientes >> temporario  
mv temporario clientes  
  
head -1 clientes > temporario  
tail -1 clientes >> temporario  
tail -7 clientes | head -6 >> temporario  
mv temporario clientes  
  
head -2 clientes > temporario  
tail -1 clientes >> temporario  
tail -6 clientes | head -5 >> temporario  
mv temporario clientes  
  
head -3 clientes > temporario  
tail -1 clientes >> temporario  
tail -5 clientes | head -4 >> temporario  
mv temporario clientes  
  
head -4 clientes > temporario  
tail -1 clientes >> temporario  
tail -4 clientes | head -3 >> temporario
```

```
mv temporario clientes

head -5 clientes > temporario
tail -1 clientes >> temporario
tail -3 clientes | head -2 >> temporario
mv temporario clientes

head -6 clientes > temporario
tail -1 clientes >> temporario
tail -2 clientes | head -1 >> temporario
mv temporario clientes
less clientes
```

Listing 11.5 – Ordenação progressiva exigida na Atividade 4.4.

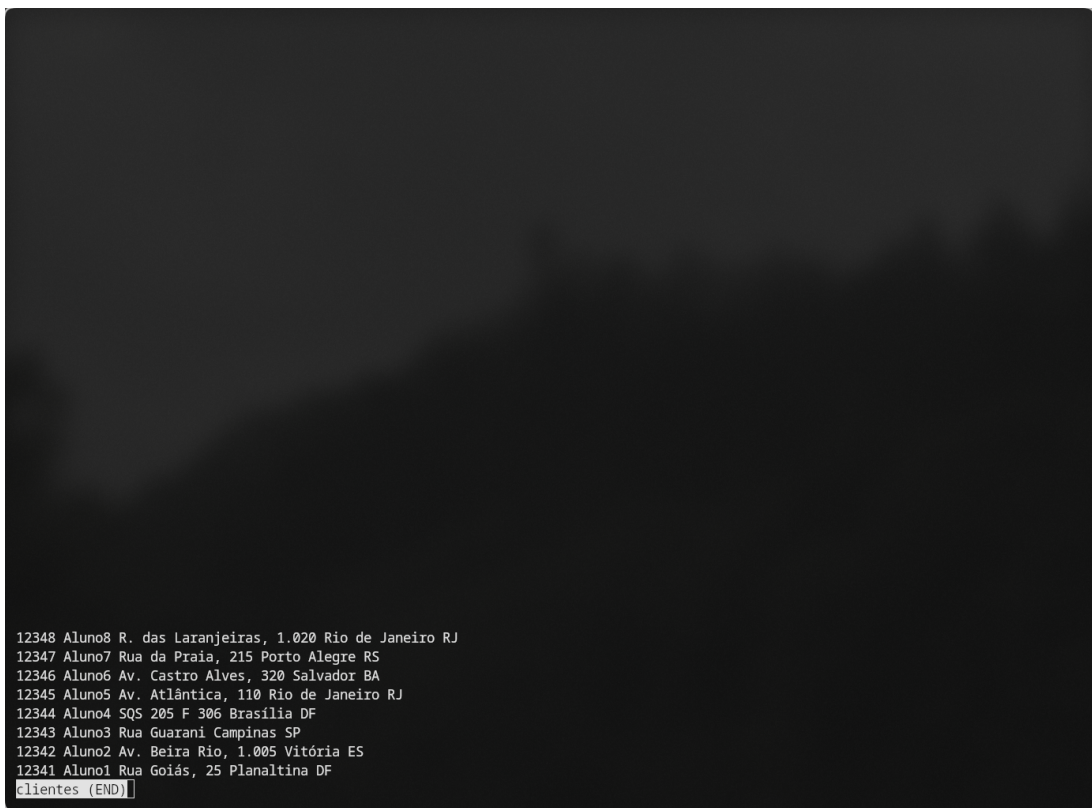


```
~ 07s
λ printf '%s\n' \
'12348 Aluno8 R. das Laranjeiras, 1.020 Rio de Janeiro RJ' \
'12347 Aluno7 Rua da Praia, 215 Porto Alegre RS' \
'12346 Aluno6 Av. Castro Alves, 320 Salvador BA' \
'12345 Aluno5 Av. Atlântica, 110 Rio de Janeiro RJ' \
'12344 Aluno4 SQS 205 F 306 Brasília DF' \
'12343 Aluno3 Rua Guarani Campinas SP' \
'12342 Aluno2 Av. Beira Rio, 1.005 Vitória ES' \
'12341 Aluno1 Rua Goiás, 25 Planaltina DF' > clientes

~
λ tail -1 clientes > temporario
head -7 clientes >> temporario
mv temporario clientes
head -1 clientes > temporario
tail -1 clientes >> temporario
tail -7 clientes | head -6 >> temporario
mv temporario clientes
head -2 clientes > temporario
tail -1 clientes >> temporario
tail -6 clientes | head -5 >> temporario
mv temporario clientes
head -3 clientes > temporario
tail -1 clientes >> temporario
tail -5 clientes | head -4 >> temporario
mv temporario clientes
head -4 clientes > temporario
tail -1 clientes >> temporario
tail -4 clientes | head -3 >> temporario
mv temporario clientes
head -5 clientes > temporario
tail -1 clientes >> temporario
tail -3 clientes | head -2 >> temporario
mv temporario clientes
head -6 clientes > temporario
tail -1 clientes >> temporario
tail -2 clientes | head -1 >> temporario
mv temporario clientes
less clientes

~ 28s
λ
```

Figura 12 – Etapas iniciais da ordenação progressiva do arquivo clientes na Atividade 4.4.



```
12348 Aluno8 R. das Laranjeiras, 1.020 Rio de Janeiro RJ
12347 Aluno7 Rua da Praia, 215 Porto Alegre RS
12346 Aluno6 Av. Castro Alves, 320 Salvador BA
12345 Aluno5 Av. Atlântica, 110 Rio de Janeiro RJ
12344 Aluno4 SQS 205 F 306 Brasília DF
12343 Aluno3 Rua Guarani Campinas SP
12342 Aluno2 Av. Beira Rio, 1.005 Vitória ES
12341 Aluno1 Rua Goiás, 25 Planaltina DF
clientes (END)
```

Figura 13 – Ordenação final do arquivo clientes na Atividade 4.4.

12 Conclusão

UNIX oferece a linhagem conceitual, GNU fornece ferramentas livres e Linux implementa o núcleo que, integrado em distribuições, sustenta ambientes de trabalho e servidores. O domínio de shell, pacotes e comandos transforma esses conceitos em prática verificável. A finalização das atividades exige as páginas identificadas e capturas produzidas pelo grupo.

Referências

Apple. *Kernel Architecture Overview*. 2013. Disponível em: <<https://developer.apple.com/library/archive/documentation/Darwin/Conceptual/KernelProgramming/Architecture/Architecture.html>>. Acesso em: 23 set. 2026.

Free Software Foundation. *About the GNU Operating System*. 2026. Disponível em: <<https://www.gnu.org/gnu/gnu.html>>. Acesso em: 23 set. 2026.

Linux Kernel Organization. *About Linux Kernel*. 2024. Disponível em: <<https://www.kernel.org/linux.html>>. Acesso em: 23 set. 2026.

Microsoft. *Overview of Windows components*. 2026. Disponível em: <<https://learn.microsoft.com/windows-hardware/drivers/kernel/overview-of-windows-components>>. Acesso em: 23 set. 2026.

PEIXOTO, A. M.; BONFANTE, A. G. *Introdução ao Linux*. Rio de Janeiro: RNP/Escola Superior de Redes, 2013. ISBN 978-85-63630-19-3.